

Từ Bigram thống kê đến Transformer decoder-only

Phân tích kiến trúc dưới các điều kiện kiểm soát về dữ liệu, tokenizer, seed và ngân sách token huấn luyện

Câu hỏi nghiên cứu. Trong điều kiện cùng corpus, tokenizer, cách chia dữ liệu, tập seed và ngân sách token huấn luyện, việc chuyển từ bigram thống kê sang neural bigram, causal self-attention và Transformer decoder-only làm thay đổi như thế nào, và vì sao, đối với: (i) chất lượng dự đoán token kế tiếp; (ii) khả năng khai thác phụ thuộc xa; (iii) hiệu quả sử dụng tham số và tổng quát hóa; (iv) runtime huấn luyện và suy luận; và (v) bộ nhớ khi context length và quy mô mô hình tăng?

Tác giả: Nguyễn Văn Duy

Loại tài liệu: Báo cáo phân tích + giao thức thực nghiệm tái lập

Phiên bản: 1.0 - Tháng 6 năm 2026

Ngôn ngữ: Tiếng Việt

Tóm tắt

Báo cáo này xây dựng một khung phân tích có kiểm soát cho quỹ đạo kiến trúc *bigram thống kê* $\rightarrow$ *neural bigram* $\rightarrow$ *causal self-attention* $\rightarrow$ *Transformer decoder-only*. Trọng tâm không phải là khẳng định một chuỗi kết quả thực nghiệm chưa được đo, mà là phân biệt rõ ba tầng bằng chứng: (1) điều suy ra trực tiếp từ cấu trúc xác suất và độ phức tạp; (2) giả thuyết có cơ sở từ tài liệu; và (3) kết luận chỉ được phép đưa ra sau thí nghiệm tái lập. Bản sửa đổi do đó loại bỏ các giá trị perplexity và throughput giả định vốn có thể bị đọc nhầm như kết quả đã quan sát.

Phân tích cho thấy neural bigram vẫn có receptive field đúng một token nhưng thay bằng chuyển tiếp rời rạc bằng một tham số hóa hạng thấp qua embedding và output head, nhờ đó có thể chia sẻ thống kê giữa các token tương tự. Causal self-attention tạo bước ngoặt về khả năng truy cập ngữ cảnh: mỗi vị trí có thể tổng hợp thông tin từ toàn bộ tiền tố trong một lớp, nhưng điều này không đồng nghĩa mô hình thực sự sử dụng hiệu quả toàn bộ context. Transformer decoder-only bổ sung chiều sâu, residual stream, multi-head attention và MLP để tái ngữ cảnh hóa biểu diễn nhiều lần, đổi lại chi phí huấn luyện và bộ nhớ activation tăng mạnh theo context length.

Báo cáo đề xuất ba chế độ so sánh bổ sung nhau: *token-matched*, *parameter-matched* và *compute-matched*. Không thể đồng thời giữ cố định số token, số tham số, FLOPs và wall-clock cho các kiến trúc khác nhau; vì vậy, một kết luận học thuật nghiêm ngặt phải nêu rõ đang trả lời câu hỏi hiệu quả theo nguồn lực nào. Giao thức thực nghiệm bao gồm cấu hình dữ liệu, tokenizer, seed, metric chất lượng, context ablation, profiling runtime/bộ nhớ, phân tích thống kê, checklist tái lập và các đe dọa tới độ hợp lệ.

Từ khóa: language modeling; statistical bigram; neural bigram; causal self-attention; Transformer decoder-only; context length; runtime; memory; reproducibility.

Abstract

This report develops a controlled analytical framework for the architectural trajectory from statistical bigrams to neural bigrams, causal self-attention, and decoder-only Transformers. It explicitly separates structural consequences, literature-grounded hypotheses, and conclusions that require empirical measurement. The report argues that the main transitions correspond to three distinct mechanisms: statistical sharing through distributed representations, full-prefix access through causal attention, and deep contextual composition through stacked Transformer blocks. It further proposes complementary token-matched, parameter-matched, and compute-matched protocols, because these resource constraints cannot all be held constant simultaneously across heterogeneous architectures. A reproducible experimental design, complexity derivations, evaluation metrics, statistical analysis plan, and validity threats are provided.

Contents

Tóm tắt	i
Abstract	i
Danh mục hình	iv
Danh mục bảng	iv
Ký hiệu và quy ước	v
1 Giới thiệu	1
1.1 Bối cảnh và vấn đề nghiên cứu	1
1.2 Câu hỏi nghiên cứu và các câu hỏi con	1
1.3 Phạm vi và loại bằng chứng	2
1.4 Đóng góp của báo cáo	2
2 Nền tảng lý thuyết và định nghĩa mô hình	2
2.1 Bigram thống kê	2
2.2 Neural bigram	3
2.3 Causal self-attention một khối	3
2.4 Transformer decoder-only	4
3 Phân tích theo năm trục nghiên cứu	5
3.1 Chất lượng dự đoán token kế tiếp	5
3.2 Khả năng khai thác phụ thuộc xa	5
3.3 Hiệu quả tham số và tổng quát hóa	5
3.4 Runtime huấn luyện và suy luận	6
3.5 Bộ nhớ khi context và quy mô tăng	6
3.6 Tổng hợp định tính	7
4 Độ phức tạp tính toán và các phép tính minh họa	7
4.1 Quy ước MAC và FLOPs	7
4.2 Chi phí theo kiến trúc	8
4.3 Ví dụ tăng chi phí theo context	8
4.4 Ví dụ KV cache	8
5 Thiết kế thí nghiệm công bằng và tái lập	9
5.1 Tại sao một protocol duy nhất là chưa đủ	9
5.2 Dữ liệu và tokenizer	9
5.3 Cấu hình mô hình đề xuất	9
5.4 Huấn luyện và tuning	10
5.5 Seed và tái lập	10

6	Metric, giả thuyết và phân tích thống kê	11
6.1	Metric chính và phụ	11
6.2	Giả thuyết tiền đăng ký	12
6.3	Ước lượng độ bất định	12
7	Ma trận kết quả cần báo cáo	12
7.1	Bảng kết quả chất lượng	12
7.2	Bảng kết quả hệ thống	13
8	Diễn giải kết quả và điều kiện kết luận	13
8.1	Khi nào có thể kết luận neural bigram tốt hơn?	13
8.2	Khi nào có thể kết luận mô hình dùng context xa?	13
8.3	Khi nào có thể kết luận Transformer hiệu quả hơn?	13
9	Đe dọa tới độ hợp lệ	14
9.1	Độ hợp lệ nội tại	14
9.2	Độ hợp lệ cấu trúc	14
9.3	Độ hợp lệ ngoại suy	14
9.4	Độ hợp lệ hệ thống	14
10	Kế hoạch triển khai đề xuất	14
10.1	Giai đoạn 1 - Pilot correctness	14
10.2	Giai đoạn 2 - Architecture ablation	15
10.3	Giai đoạn 3 - Scaling và systems	15
10.4	Tiêu chí hoàn thành	15
11	Kết luận	15
A	Công thức tham số tham khảo	16
B	Checklist tái lập	16
C	Cấu trúc thư mục thí nghiệm đề xuất	16
	Tài liệu tham khảo	18

List of Figures

1	Bốn mốc kiến trúc và cơ chế mới được đưa vào ở từng bước.	1
2	Causal mask bảo đảm vị trí i không truy cập token tương lai $j > i$	4
3	Tăng trưởng lý thuyết tương đối theo context length. Đường cong minh họa độ phức tạp, không phải benchmark wall-clock.	7
4	Pipeline thực nghiệm từ dữ liệu bất biến đến phân tích thống kê.	11

List of Tables

1	Tác động cơ chế dự kiến của từng bước chuyển kiến trúc.	7
2	Độ phức tạp xấp xỉ cho forward pass.	8
3	Phép tính minh họa cho một Transformer 12 lớp, $d = 768$	8
4	Cấu hình pilot minh họa; tham số cuối phải được tính từ code thực tế.	10
5	Các giả thuyết có thể kiểm định.	12
6	Template báo cáo chất lượng.	13
7	Template báo cáo runtime và bộ nhớ tại từng context length.	13

Ký hiệu và quy ước

Ký hiệu	Ý nghĩa
V	Kích thước vocabulary.
d	Kích thước biểu diễn ẩn hoặc embedding.
L	Context length hoặc số token trong một sequence.
B	Batch size theo số sequence.
h	Số attention head; $d_h = d/h$ là chiều của mỗi head.
n_ℓ	Số block Transformer.
N	Số token của corpus hoặc số token đã xử lý, tùy ngữ cảnh.
MAC	Một phép nhân-tích lũy; nếu quy đổi sang FLOPs theo quy ước nhân và cộng là hai phép, cần nhân xấp xỉ hai.
PPL	Perplexity, $PPL = \exp(\text{cross-entropy trung bình})$.
ECL	Effective context length: độ dài ngữ cảnh thực tế còn tạo cải thiện đo được, không phải receptive field lý thuyết.

1 Giới thiệu

1.1 Bối cảnh và vấn đề nghiên cứu

Mô hình ngôn ngữ tự hồi quy ước lượng xác suất của token kế tiếp theo tiền tố đã quan sát:

$$P(x_{1:T}) = \prod_{t=1}^T P(x_t | x_{<t}). \quad (1)$$

Sự khác biệt giữa các kiến trúc không nằm ở mục tiêu xác suất này, mà ở cách chúng nén lịch sử $x_{<t}$ thành thông tin hữu ích cho dự đoán. Bigram thống kê áp đặt giả định Markov bậc một; neural bigram giữ nguyên phạm vi ngữ cảnh nhưng thay cách tham số hóa; causal self-attention mở quyền truy cập toàn bộ tiền tố; còn Transformer decoder-only tái xử lý tiền tố qua nhiều tầng phi tuyến.

Bản chất của câu hỏi nghiên cứu vì vậy không phải đơn giản là “mô hình nào tốt hơn”, mà là: *thay đổi cơ chế biểu diễn và truyền thông tin làm thay đổi chất lượng, tổng quát hóa và chi phí hệ thống ra sao khi các yếu tố ngoại lai được kiểm soát?* Các công trình nền tảng về neural language modeling, self-attention và GPT cung cấp cơ sở cơ chế cho quỹ đạo này [2, 3, 4, 5]. Tuy nhiên, một nghiên cứu thực nghiệm còn phải tách tác động của kiến trúc khỏi tác động của số tham số, compute, dữ liệu và tối ưu hóa.

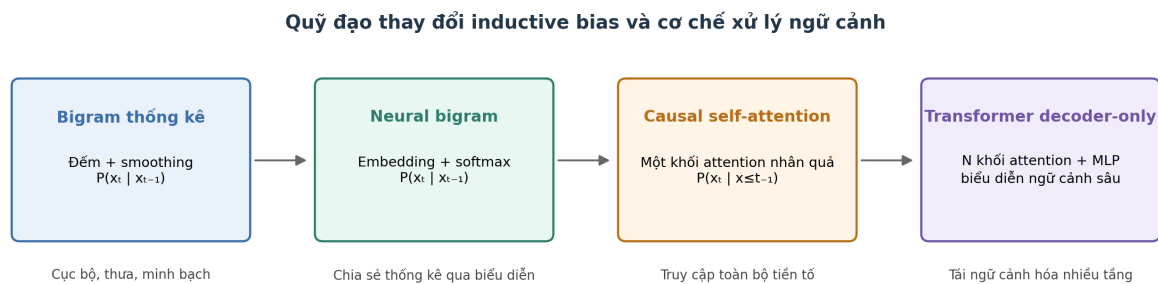


Figure 1. Bốn mốc kiến trúc và cơ chế mới được đưa vào ở từng bước.

1.2 Câu hỏi nghiên cứu và các câu hỏi con

Câu hỏi trung tâm được tách thành năm câu hỏi con có thể đo lường:

1. **Chất lượng dự đoán:** cross-entropy, perplexity và top- k accuracy thay đổi thế nào?
2. **Phụ thuộc xa:** mô hình có cải thiện khi được cung cấp thêm context không, và cải thiện còn tồn tại đến khoảng cách nào?
3. **Hiệu quả tham số và tổng quát hóa:** cùng số tham số hoặc cùng compute, kiến trúc nào đạt loss tốt hơn và ổn định hơn trên dữ liệu chưa thấy?
4. **Runtime:** training throughput, prefill throughput, decode throughput và latency/token tăng giảm ra sao?
5. **Bộ nhớ:** parameter state, optimizer state, activation và KV cache tăng theo L , d và n_ℓ như thế nào?

1.3 Phạm vi và loại bằng chứng

Giới hạn diễn giải

Tài liệu này là **báo cáo phân tích lý thuyết và thiết kế thực nghiệm**, chưa phải báo cáo kết quả huấn luyện. Mọi phát biểu dạng “dự kiến”, “có khả năng” hoặc “giả thuyết” phải được kiểm chứng bằng dữ liệu. Không có giá trị perplexity, tokens/s hay mức sử dụng bộ nhớ nào được trình bày như số đo thực nghiệm nếu chưa chạy benchmark.

Ba loại phát biểu được dùng xuyên suốt:

- **Hệ quả cấu trúc:** suy ra từ đồ thị tính toán hoặc công thức, ví dụ bigram chỉ điều kiện hóa trên một token.
- **Giả thuyết có cơ sở:** được hỗ trợ bởi lý thuyết và tài liệu, nhưng hướng và độ lớn hiệu ứng còn phụ thuộc dữ liệu và cấu hình.
- **Kết luận thực nghiệm:** chỉ hợp lệ sau khi báo cáo số đo, độ bất định, seed và môi trường thực thi.

1.4 Đóng góp của báo cáo

Bản báo cáo hoàn thiện đóng góp bốn thành phần: (i) định nghĩa chính xác bốn baseline; (ii) phân tích năm trục chất lượng - ngữ cảnh - tham số - runtime - bộ nhớ; (iii) thiết kế so sánh công bằng theo nhiều chế độ nguồn lực; và (iv) protocol tái lập có thể chuyển trực tiếp thành kế hoạch triển khai MiniGPT.

2 Nền tảng lý thuyết và định nghĩa mô hình

2.1 Bigram thống kê

Với giả định Markov bậc một,

$$P(x_t \mid x_{<t}) \approx P(x_t \mid x_{t-1}). \quad (2)$$

Ước lượng cực đại khả năng là

$$\hat{P}_{\text{MLE}}(v \mid u) = \frac{C(u, v)}{C(u)}. \quad (3)$$

Do dữ liệu thưa, ước lượng MLE gán xác suất bằng không cho các cặp chưa thấy. Một baseline học thuật nghiêm túc nên dùng smoothing mạnh, chẳng hạn interpolated modified Kneser-Ney, thay vì add-one smoothing. Chen và Goodman cho thấy hiệu quả smoothing phụ thuộc đáng kể vào kích thước dữ liệu và n-gram order [1]. KenLM là một lựa chọn triển khai thực dụng cho truy vấn n-gram hiệu quả [15].

Dạng khái quát của interpolated Kneser-Ney bigram có thể viết:

$$P_{\text{KN}}(v \mid u) = \frac{\max(C(u, v) - D(C(u, v)), 0)}{C(u)} + \lambda(u)P_{\text{cont}}(v), \quad (4)$$

trong đó $D(\cdot)$ là discount, $\lambda(u)$ phân phối lại xác suất đã chiết khấu, và P_{cont} ưu tiên token xuất hiện sau nhiều lịch sử khác nhau.

Đặc tính chính. Training là đếm và ước lượng xác suất, gần tuyến tính theo số token. Mô hình minh bạch và mạnh khi tín hiệu chủ yếu cục bộ, nhưng không có cơ chế học similarity giữa các token lịch sử và không thể sử dụng x_{t-2} hay xa hơn.

2.2 Neural bigram

Neural bigram dùng embedding của token trước và output head:

$$e_{t-1} = E[x_{t-1}] \in \mathbb{R}^d, \quad (5)$$

$$z_t = W e_{t-1} + b, \quad (6)$$

$$P(x_t = v \mid x_{t-1}) = \text{softmax}(z_t)_v. \quad (7)$$

Với embedding đầu vào và output matrix không buộc chung trọng số, số tham số xấp xỉ

$$P_{\text{NB}} \approx 2Vd + V. \quad (8)$$

Nếu weight tying được dùng, số tham số vocabulary giảm gần một nửa, nhưng đây là một thay đổi phương pháp phải áp dụng nhất quán hoặc báo cáo riêng.

Điểm sâu hơn của neural bigram là ma trận logit chuyển tiếp có dạng $WE^\top$ (bỏ qua bias), nên hạng của nó không vượt quá d . Mô hình vì thế thực hiện một dạng **factorization hạng thấp của quan hệ chuyển tiếp**. Điều này tạo chia sẻ thống kê và có thể giảm overfitting khi $d \ll V$, nhưng cũng có thể underfit nếu phân phối bigram cần hạng cao. Distributed representations là cơ chế trung tâm để chống curse of dimensionality trong neural language modeling [2].

Kết luận cơ chế 1

Chuyển từ bigram thống kê sang neural bigram **không mở rộng receptive field**. Nó thay đổi cách tham số hóa: từ các hàng xác suất gần độc lập sang một không gian biểu diễn dùng chung. Vì vậy, cải thiện nếu có phải được quy cho *statistical sharing*, không phải long-range context.

2.3 Causal self-attention một khối

Để cô lập đóng góp của attention, báo cáo định nghĩa baseline trung gian là một khối gồm token embedding, positional encoding, masked multi-head self-attention, residual connection, layer normalization và LM head; MLP có thể được loại bỏ trong ablation chính hoặc đưa vào một ablation phụ. Với một head:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (9)$$

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right), \quad (10)$$

$$H = AVW_O, \quad (11)$$

trong đó $M_{ij} = -\infty$ nếu $j > i$ và bằng 0 nếu $j \leq i$.

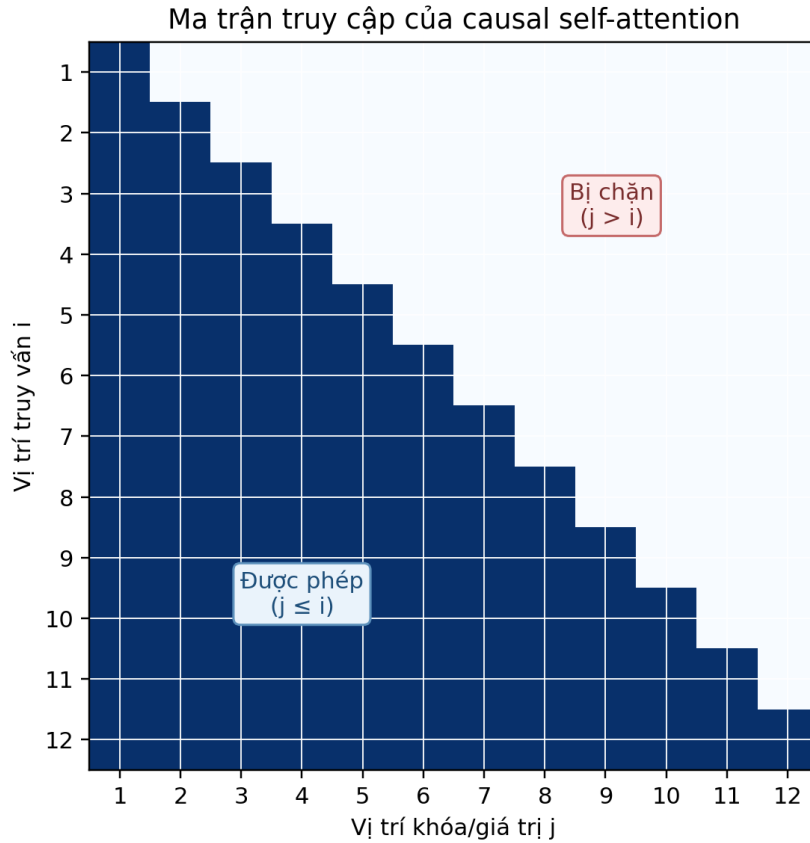


Figure 2. Causal mask bảo đảm vị trí i không truy cập token tương lai $j > i$.

Self-attention cho phép đường truyền giữa hai vị trí bất kỳ trong một lớp có độ dài hằng số theo bảng phân tích của Transformer gốc [3]. Tuy nhiên, **receptive field lý thuyết không đồng nhất với effective context**. Mô hình có thể truy cập 512 token nhưng chỉ thu được cải thiện đáng kể từ vài chục token gần nhất. Positional information cũng là điều kiện cần để mô hình phân biệt trật tự và khoảng cách.

2.4 Transformer decoder-only

Một block pre-norm tiêu biểu là

$$\tilde{h}^{(\ell)} = h^{(\ell)} + \text{MHSA}(\text{LN}(h^{(\ell)})), \quad (12)$$

$$h^{(\ell+1)} = \tilde{h}^{(\ell)} + \text{MLP}(\text{LN}(\tilde{h}^{(\ell)})). \quad (13)$$

Nhiều block cho phép mô hình lặp lại ba thao tác: chọn thông tin liên quan, biến đổi phi tuyến theo vị trí, và ghi kết quả vào residual stream. GPT sử dụng stack decoder-only với masked self-attention cho mục tiêu language modeling tự hồi quy [4, 5].

Kết luận cơ chế 2

Chuyển từ một khối attention sang Transformer nhiều tầng không chủ yếu mở rộng receptive field lý thuyết - cả hai đã thấy toàn bộ tiền tố. Lợi ích kỳ vọng đến từ **độ sâu biểu diễn**: nhiều vòng truy hồi, tổng hợp và composition của đặc trưng ngữ cảnh.

3 Phân tích theo năm trục nghiên cứu

3.1 Chất lượng dự đoán token kế tiếp

Bigram thống kê có inductive bias mạnh và variance thấp khi dữ liệu nhỏ, đặc biệt nếu smoothing tốt. Neural bigram có thể giảm sparsity nhờ factorization, nhưng không có bảo đảm luôn vượt modified Kneser-Ney: một embedding quá nhỏ có thể underfit, trong khi một embedding quá lớn có thể mất lợi thế regularization.

Causal self-attention có tiềm năng giảm loss khi token đúng phụ thuộc vào dấu hiệu ngoài token liền trước. Mức cải thiện sẽ nhỏ nếu corpus bị xáo trộn theo câu hoặc nếu task gần như Markov bậc một. Transformer nhiều tầng dự kiến tiếp tục giảm loss khi có đủ dữ liệu và tối ưu hóa ổn định, nhưng dưới ngân sách token quá thấp, mô hình sâu có thể bị undertrained. Scaling-law studies cho thấy loss liên hệ có hệ thống với model size, data và compute; nghiên cứu Chinchilla nhấn mạnh rằng tăng tham số mà không tăng đủ token là phân bổ compute kém tối ưu [6, 7].

3.2 Khả năng khai thác phụ thuộc xa

Receptive field của hai bigram bằng một. Receptive field của causal attention và decoder-only bằng toàn bộ tiền tố trong giới hạn L . Vì vậy, phép đo đúng không chỉ hỏi “mô hình có thể nhìn bao xa”, mà hỏi “thêm context xa giúp loss bao nhiêu”.

Một phép đo trực tiếp là context ablation:

$$\Delta_k = \text{CE}(x_t \mid x_{t-k:t-1}) - \text{CE}(x_t \mid x_{t-L:t-1}), \quad (14)$$

trong đó $k \in \{1, 8, 32, 128, 512\}$. Với bigram, Δ_k phải xấp xỉ 0 cho mọi $k \geq 1$. Với attention, đường cong Δ_k cho biết context hiệu dụng. LAMBADA được thiết kế để yêu cầu broad discourse context và có thể dùng như probe bổ sung, không nên thay thế evaluation trên chính corpus nghiên cứu [13].

3.3 Hiệu quả tham số và tổng quát hóa

Có ba nghĩa khác nhau của “hiệu quả tham số”:

1. loss tốt hơn tại cùng tổng số tham số;
2. loss tốt hơn tại cùng tham số không tính embedding;
3. loss tốt hơn trên mỗi byte bộ nhớ hoặc mỗi đơn vị compute.

Cần công bố định nghĩa đang dùng. Với vocabulary lớn, embedding và output head có thể chiếm phần lớn tham số ở mô hình nhỏ; so sánh tổng tham số có thể che khuất phần backbone. Do đó nên báo cáo cả *total parameters* và *non-embedding parameters*.

Neural bigram chia sẻ thống kê nhưng chỉ biểu diễn quan hệ chuyển tiếp bậc một. Attention chia sẻ cùng các projection trên mọi vị trí, còn Transformer dùng lại cấu trúc block trên toàn sequence. Tuy nhiên, khả năng tổng quát hóa không đến miễn phí: mô hình có năng lực lớn hơn cũng nhạy với data quality, regularization và undertraining.

3.4 Runtime huấn luyện và suy luận

Bigram thống kê có training gần $O(N)$ và lookup rất rẻ. Neural bigram gần như không có backbone theo chiều sequence, nhưng dense output softmax cần khoảng $O(Vd)$ MAC mỗi token. Ở mô hình nhỏ và vocabulary lớn, output head có thể chi phối runtime, khiến chênh lệch giữa neural bigram và attention nông nhỏ hơn trực giác chỉ nhìn backbone.

Trong training teacher-forced, self-attention song song hóa theo vị trí nhưng tạo ma trận $L \times L$. Transformer nhiều tầng nhân chi phí theo n_ℓ . Trong inference tự hồi quy, phải tách hai pha:

- **Prefill:** xử lý prompt song song; hành vi gần training forward.
- **Decode:** sinh từng token; KV cache tránh tính lại toàn bộ prefix nhưng phải đọc cache tăng theo L .

FlashAttention giữ nguyên attention chính xác nhưng giảm IO và không materialize một số tensor trung gian lớn, cải thiện wall-clock và memory footprint mà không thay đổi mục tiêu mô hình [8]. PagedAttention giải quyết phân mảnh và quản lý KV cache ở serving, vì vậy không nên nhầm đây là thay đổi kiến trúc học [9].

3.5 Bộ nhớ khi context và quy mô tăng

Tổng footprint gồm bốn nhóm:

$$M_{\text{total}} = M_{\text{params}} + M_{\text{optimizer}} + M_{\text{activations}} + M_{\text{KV}}. \quad (15)$$

Statistical bigram chủ yếu dùng sparse counts/probabilities. Neural models có parameter state; khi dùng AdamW, optimizer thường giữ thêm moment states, làm bộ nhớ training lớn hơn nhiều so với chỉ trọng số [14]. Standard attention tạo activation theo $O(BhL^2)$ cho score/probability tensors trong triển khai ngây thơ. Ở inference, KV cache cho full multi-head attention là

$$M_{\text{KV}} = 2B n_\ell L d s, \quad (16)$$

trong đó hệ số 2 là K và V, s là số byte mỗi phần tử. Với GQA/MQA, thay d của KV bằng $n_{kv}d_h$, nên cache giảm theo tỷ lệ số KV head [10].

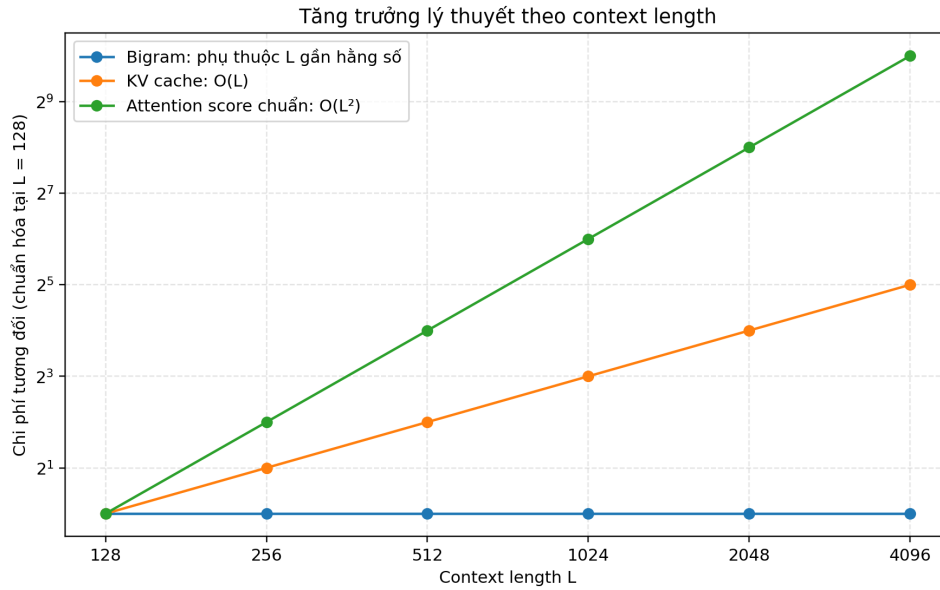


Figure 3. Tăng trưởng lý thuyết tương đối theo context length. Đường cong minh họa độ phức tạp, không phải benchmark wall-clock.

3.6 Tổng hợp định tính

Table 1. Tác động cơ chế dự kiến của từng bước chuyển kiến trúc.

Bước chuyển	Chất lượng	Ngữ cảnh xa	Hiệu quả tham số	Chi phí hệ thống
Statistical bigram → neural bigram	Có thể tốt hơn trên dữ liệu thưa; không bảo đảm nếu factorization underfit	Không đối: trực tiếp chỉ 1 token	Chia sẻ thống kê; state từ V^2 tiềm năng về $O(Vd)$	Thêm SGD và dense softmax; context cost gần hằng
Neural bigram → causal attention	Tốt hơn khi có cues ngoài token trước	Bước ngoặt: truy cập toàn tiền tố	Projection dùng chung theo vị trí; cần positional signal	Training xuất hiện L^2 ; inference cần KV cache
Causal attention → decoder-only sâu	Tốt hơn nếu đủ data/compute và tối ưu ổn định	Receptive field không đối, nhưng ECL có thể tăng	Composition nhiều tầng; tham số backbone tăng	Runtime và activation nhân theo số lớp; decode memory tăng tuyến tính theo $n_\ell L$

4 Độ phức tạp tính toán và các phép tính minh họa

4.1 Quy ước MAC và FLOPs

Các công thức dưới đây đếm MAC cho một forward pass, bỏ qua hằng số nhỏ, layer normalization, bias và softmax elementwise. Một số tài liệu gọi các biểu thức này là FLOPs; nếu một phép nhân và một phép cộng được tính riêng, số FLOPs xấp xỉ gấp đôi. Báo cáo benchmark phải ghi rõ quy ước.

4.2 Chi phí theo kiến trúc

Table 2. Độ phức tạp xấp xỉ cho forward pass.

Kiến trúc	MAC training/eval teacher-forced	MAC mỗi token decode	Bộ nhớ phụ thuộc context
Statistical bigram	$O(N)$ cho toàn bộ quá trình đếm	Lookup/backoff gần $O(1)$	Không có activation L^2
Neural bigram	Xấp xỉ LdV cho output head	Xấp xỉ dV	Hidden trong batch $O(BLd)$
Một lớp MHSA	$4Ld^2 + 2L^2d$ cộng LM head	Với cache: $O(d^2 + Ld)$ cộng LM head	Score chuẩn $O(BhL^2)$; KV $O(BLd)$
Decoder-only	$n_\ell(12Ld^2 + 2L^2d)$ cộng LM head	$O(n_\ell(d^2 + Ld))$ cộng LM head	Activation tăng theo lớp; KV $O(Bn_\ell Ld)$

Hệ số $12Ld^2$ của một block tiêu chuẩn gồm $4Ld^2$ cho Q, K, V và output projection, cùng $8Ld^2$ cho MLP có expansion ratio 4. Hai phép nhân attention chính tạo $2L^2d$ MAC.

4.3 Ví dụ tăng chi phí theo context

Xét decoder-only với $d = 768$, $h = 12$, $n_\ell = 12$, batch $B = 8$, fp16, chỉ tính backbone. MAC forward mỗi token là

$$\frac{n_\ell(12Ld^2 + 2L^2d)}{L} = n_\ell(12d^2 + 2Ld). \quad (17)$$

Table 3. Phép tính minh họa cho một Transformer 12 lớp, $d = 768$.

L	Forward MAC/token	Bộ nhớ score $Bhn_\ell L^2$ ở fp16
128	87.3 triệu	0.035 GiB
512	94.4 triệu	0.563 GiB
2048	122.7 triệu	9.00 GiB

Các con số score-memory chỉ minh họa một tensor score mỗi lớp; training thực tế còn cần activations khác và backward state. FlashAttention thay đổi đáng kể footprint triển khai, nhưng không xóa phần tính toán L^2 của exact attention [8].

4.4 Ví dụ KV cache

Với full multi-head attention, batch 1, fp16:

- $n_\ell = 24$, $d = 1024$, $L = 8192$: $M_{KV} = 0.75$ GiB.
- $n_\ell = 80$, $d = 8192$, $L = 8192$: $M_{KV} = 20$ GiB.

Đây chỉ là cache K và V, chưa gồm trọng số, workspace và allocator overhead. MQA/GQA và paging làm giảm hoặc quản lý footprint tốt hơn, nhưng cần báo cáo rõ cấu hình head và serving engine.

5 Thiết kế thí nghiệm công bằng và tái lập

5.1 Tại sao một protocol duy nhất là chưa đủ

Cùng token budget không đồng nghĩa cùng compute. Transformer sâu thực hiện nhiều MAC hơn cho mỗi token so với neural bigram. Ngược lại, cùng compute có thể buộc mô hình sâu nhìn ít token hơn. Cùng tham số cũng không bảo đảm cùng runtime do cấu trúc tensor và memory traffic khác nhau.

Ba chế độ so sánh cần báo cáo

Track A - Token-matched: cùng số token train; trả lời kiến trúc học được gì từ cùng lượng dữ liệu quan sát.

Track B - Parameter-matched: cùng tổng tham số hoặc cùng non-embedding parameters; trả lời inductive bias nào sử dụng tham số hiệu quả hơn.

Track C - Compute-matched: cùng tổng MAC/FLOPs hoặc GPU-hours; trả lời kiến trúc nào hiệu quả hơn theo ngân sách tính toán.

Không thể đồng thời giữ cả ba hoàn toàn bằng nhau. Kết luận cuối phải gắn với track cụ thể, thay vì tuyên bố một kiến trúc “tốt hơn” tuyệt đối.

5.2 Dữ liệu và tokenizer

Một pilot để tái lập có thể dùng WikiText-103, được giới thiệu để đánh giá language modeling với context dài và vocabulary thực tế hơn [12]. Tokenizer nên được train đúng một lần trên train split, sau đó đóng băng cho mọi mô hình. SentencePiece là lựa chọn phù hợp vì hoạt động trực tiếp trên raw text và độc lập ngôn ngữ [11].

Giao thức dữ liệu tối thiểu:

1. lưu checksum của raw files, manifest và tokenizer model;
2. split theo document trước khi tạo sequence để tránh leakage;
3. cố định normalization, vocabulary size, special tokens và packing;
4. không để sequence vượt qua ranh giới document trừ khi đó là chủ ý đã đăng ký trước;
5. lưu tokenized dataset dưới dạng bất biến, dùng chung cho mọi kiến trúc.

5.3 Cấu hình mô hình đề xuất

Để không trộn “attention” với “nhiều tham số hơn”, nên có ít nhất hai family cấu hình.

Table 4. Cấu hình pilot minh họa; tham số cuối phải được tính từ code thực tế.

Mô hình	Đầu vào	Backbone	Context train	Ghi chú đối sánh
Statistical bigram	Token ID trước	MKN sparse	1	Baseline xác suất mạnh
Neural bigram	Token trước, $d \in \{192, 384\}$	Embedding + linear head	1	Báo cáo tied và untied riêng nếu có
Causal-attention ablation	Chuỗi, $d = 192$, $h = 4$	1 MHSA block, không MLP trong ablation chính	512	Positional encoding bắt buộc
Decoder-only tiny	Chuỗi, $d = 192$, $h = 4$	4 blocks, MLP ratio 4	512	Xấp xỉ parameter-matched với head vocabulary lớn
Decoder-only small	Chuỗi, $d = 384$, $h = 6$	8 blocks	512	Dùng cho scaling sweep, không phải đối sánh chính

5.4 Huấn luyện và tuning

Các neural baseline phải dùng cùng framework và precision. AdamW là lựa chọn mặc định hợp lý; weight decay cần được tách khỏi gradient update theo định nghĩa của AdamW [14]. Tuy nhiên, “cùng learning rate” không nhất thiết công bằng vì độ sâu và normalization khác nhau. Phương án tốt hơn là dùng cùng *quy trình tuning*: cùng search space, cùng số trial, cùng validation budget.

Cấu hình cần khóa và log:

- optimizer, betas, epsilon, weight decay, gradient clipping;
- learning-rate schedule, warmup theo số token, không chỉ số step;
- global tokens/update và gradient accumulation;
- initialization, dropout, positional encoding, weight tying;
- precision, kernel attention, compiler flags và gradient checkpointing;
- early stopping rule hoặc số token dừng cố định.

5.5 Seed và tái lập

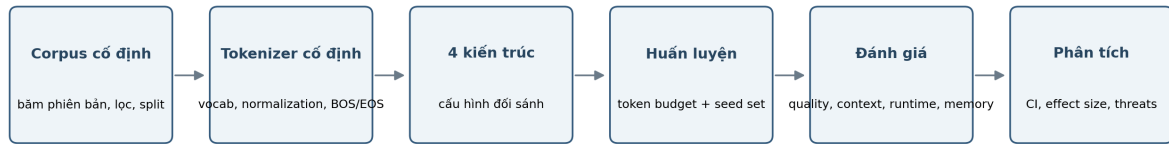
Một seed duy nhất không đủ để ước lượng độ bất định. Nên dùng cùng một tập seed, ví dụ {17, 29, 43, 71, 101}, cho mọi neural model. Statistical bigram thường xác định sau preprocessing, nhưng vẫn phải khóa thứ tự dữ liệu nếu implementation có bước ngẫu nhiên.

PyTorch lưu ý rằng tính tái lập hoàn toàn không được bảo đảm giữa release, platform, CPU và GPU, và deterministic operations có thể chậm hơn [16]. Vì vậy cần lưu:

- commit hash, lockfile, container image, CUDA/cuDNN và driver;

- GPU model, số GPU, distributed backend và environment variables;
- seed Python, NumPy, PyTorch CPU/CUDA và DataLoader workers;
- trạng thái deterministic algorithms và benchmark flags.

Giao thức thực nghiệm tái lập và có thể kiểm toán



Mỗi thay đổi kiến trúc phải được tách khỏi ảnh hưởng của dữ liệu, tokenizer, số token, tham số và compute.

Figure 4. Pipeline thực nghiệm từ dữ liệu bất biến đến phân tích thống kê.

6 Metric, giả thuyết và phân tích thống kê

6.1 Metric chính và phụ

Primary endpoint nên là test cross-entropy trên cùng tokenization, vì perplexity chỉ là biến đổi mũ của cùng đại lượng. Metric phụ gồm top-1/top-5 next-token accuracy, bits-per-token, calibration error và loss theo tần suất token.

Đối với phụ thuộc xa:

- context ablation curve;
- loss theo khoảng cách đến evidence token nếu có annotation/probe;
- LAMBADA accuracy/perplexity như external probe [13];
- đánh giá trên contiguous documents, không chỉ sentence-shuffled data.

Đối với hệ thống:

- training: effective tokens/s, step time, utilization, energy nếu đo được;
- inference: time-to-first-token, prefill tokens/s, decode tokens/s, p50/p95 latency;
- memory: peak allocated, peak reserved và phân rã parameter, optimizer, activation, KV cache;
- tất cả benchmark có warm-up, synchronization và ít nhất 30 lần lặp đo.

6.2 Giả thuyết tiền đăng ký

Table 5. Các giả thuyết có thể kiểm định.

ID	Giả thuyết	Cơ chế	Phép kiểm tra chính
H1	Neural bigram giảm test CE so với MKN bigram trong dữ liệu đủ thừa	Low-rank sharing giữa token histories	Paired bootstrap trên sequence; mean qua seed
H2	Causal attention giảm CE rõ hơn ở token có evidence xa	Full-prefix access	Context ablation và distance-conditioned loss
H3	Decoder-only sâu giảm CE so với 1-block attention khi token budget đủ	Deep contextual composition	Track A và Track B
H4	Standard attention memory tăng xấp xỉ bậc hai theo L trong triển khai không flash	$L \times L$ score/probability tensors	Log-log regression peak memory theo L
H5	Decode latency và KV footprint tăng gần tuyến tính theo L khi dùng cache	Đọc và lưu K/V quá khứ	Sweep L với batch cố định

H1 không nên được viết như chân lý: modified Kneser-Ney có thể thắng neural bigram ở corpus nhỏ hoặc tuning kém. H3 cũng phụ thuộc mức undertraining. Việc tiền đăng ký điều kiện bác bỏ giúp tránh diễn giải hậu nghiệm.

6.3 Ước lượng độ bất định

Báo cáo mỗi neural configuration bằng mean $\pm$ standard deviation qua seed, đồng thời bootstrap trên test documents để tạo confidence interval cho chênh lệch loss. Vì seed thường chỉ 3-5, không nên dựa duy nhất vào kiểm định t với giả định chuẩn. Hiệu ứng nên báo cáo bằng chênh lệch absolute CE, relative PPL reduction và effect size chuẩn hóa.

Đối với runtime, dùng median và p95 thay vì chỉ mean; loại bỏ warm-up nhưng không loại outlier tùy ý. Cùng một machine phải chạy các model xen kẽ hoặc random hóa thứ tự benchmark để giảm bias nhiệt và tải hệ thống.

7 Ma trận kết quả cần báo cáo

7.1 Bảng kết quả chất lượng

Bảng dưới đây là template, cố ý để trống thay vì điền số giả định.

Table 6. Template báo cáo chất lượng.

Mô hình	Params	Train tokens	Test CE	PPL	Top-1	ECL
MKN bigram	–	–	–	–	–	1
Neural bigram	–	–	–	–	–	1
1-block causal attention	–	–	–	–	–	–
4-block decoder-only	–	–	–	–	–	–

7.2 Bảng kết quả hệ thống

Table 7. Template báo cáo runtime và bộ nhớ tại từng context length.

Mô hình / L	Train tok/s	TTFT	Decode tok/s	Peak train GiB	KV GiB	Kernel
Model A / 512	–	–	–	–	–	–
Model A / 2048	–	–	–	–	–	–
Model B / 512	–	–	–	–	–	–
Model B / 2048	–	–	–	–	–	–

8 Diễn giải kết quả và điều kiện kết luận

8.1 Khi nào có thể kết luận neural bigram tốt hơn?

Chỉ khi chênh lệch test loss ổn định qua seed và không do thay đổi tokenizer, weight tying hay tuning budget. Nếu neural bigram có PPL thấp hơn nhưng nhiều tham số hơn đáng kể, kết luận hợp lệ là “tốt hơn trong Track A”, không phải “hiệu quả tham số hơn”.

8.2 Khi nào có thể kết luận mô hình dùng context xa?

PPL tổng thể thấp hơn chưa đủ. Cần chỉ ra rằng tăng context tại evaluation cải thiện loss; hoặc cắt/hoán đổi evidence xa làm xác suất token đúng giảm. Attention visualization không phải bằng chứng nhân quả đầy đủ.

8.3 Khi nào có thể kết luận Transformer hiệu quả hơn?

Cần nêu rõ mẫu số: trên mỗi parameter, mỗi training token, mỗi FLOP, mỗi GPU-hour hay mỗi GiB serving. Transformer có thể tốt nhất về loss nhưng kém nhất về latency hoặc memory. Một nghiên cứu hệ thống tốt không ép năm trục thành một thứ hạng duy nhất.

Kết luận định tính được phép trước thí nghiệm

- Bigram thống kê và neural bigram không thể trực tiếp dùng token xa hơn một bước.
- Neural bigram thêm statistical sharing nhưng áp đặt factorization hạng thấp.
- Causal attention mở full-prefix access và đưa vào chi phí tương tác theo L^2 trong

training chuẩn.

- Transformer sâu thêm khả năng composition, đồng thời tăng compute, activation và KV cache theo số lớp.
- Độ lớn lợi ích chất lượng không thể suy ra chỉ từ công thức; nó phải được đo dưới protocol công bằng.

9 Đe dọa tới độ hợp lệ

9.1 Độ hợp lệ nội tại

- **Tuning inequity:** một model được tuning nhiều hơn model khác.
- **Parameter mismatch:** output head chiếm tỷ lệ khác nhau, gây sai diễn giải.
- **Kernel mismatch:** một model dùng fused kernel/FlashAttention, model khác dùng implementation chậm.
- **Early stopping leakage:** chọn checkpoint theo test thay vì validation.
- **Randomness:** một seed duy nhất hoặc môi trường không khóa.

9.2 Độ hợp lệ cấu trúc

Perplexity đo next-token modeling, không đồng nhất với hiểu ngôn ngữ. LAMBADA đo một dạng broad-context prediction nhưng có domain riêng. “Effective context” phụ thuộc phép đo; context ablation có thể bỏ lỡ trường hợp model dùng context để calibration chứ không đổi top-1.

9.3 Độ hợp lệ ngoại suy

Kết luận trên WikiText-103, vocabulary 32k và $L = 512$ không tự động mở rộng sang tiếng Việt, code, context 128k hay mô hình hàng tỷ tham số. Scaling laws là quan hệ thực nghiệm trong miền khảo sát, không phải định luật bảo đảm mọi kiến trúc [6, 7].

9.4 Độ hợp lệ hệ thống

Tokens/s phụ thuộc GPU, precision, batch, compiler, kernel và thermal state. Peak allocated của framework có thể khác memory vật lý do allocator/cache. Kết quả serving phụ thuộc batching policy, request-length distribution và engine; PagedAttention có thể thay đổi throughput mà không thay đổi trọng số mô hình [9].

10 Kế hoạch triển khai đề xuất

10.1 Giai đoạn 1 - Pilot correctness

1. Xây tokenized dataset nhỏ và unit test xác suất/loss.
2. Implement MKN bigram và neural bigram.
3. Kiểm tra neural bigram với $L = 1$ cho kết quả độc lập context dài.

4. Implement một block causal attention và verify mask bằng test không rò token tương lai.
5. Implement decoder-only 4 block, weight tying là config tường minh.

10.2 Giai đoạn 2 - Architecture ablation

Chạy Track A và B trên tiny configurations, ít nhất ba seed. Tạo context ablation curve và profile $L \in \{128, 256, 512, 1024\}$. Đây là giai đoạn trả lời trực tiếp cơ chế shared representation, full-prefix access và deep composition.

10.3 Giai đoạn 3 - Scaling và systems

Mở rộng d, n_ℓ và token budget; chạy Track C; so sánh standard attention với FlashAttention; profile prefill/decode và KV cache. Chỉ ở giai đoạn này mới bàn tới compute-optimal scaling và serving trade-off.

10.4 Tiêu chí hoàn thành

Một báo cáo thực nghiệm hoàn chỉnh phải kèm code commit, environment lock, data/tokenizer hashes, config files, raw logs, checkpoints hoặc ít nhất checksum, script tạo bảng/hình, và một lệnh tái chạy end-to-end.

11 Kết luận

Quỹ đạo từ statistical bigram đến decoder-only Transformer nên được hiểu như ba bước thay đổi cơ chế: **chia sẻ thống kê**, **truy cập tiền tố**, và **composition nhiều tầng**. Bigram thống kê rẻ và minh bạch nhưng bị giới hạn cứng ở context một token. Neural bigram giảm sparsity bằng một factorization học được nhưng không giải quyết phụ thuộc xa. Causal self-attention mở quyền truy cập toàn prefix, đồng thời đưa chi phí attention theo bậc hai của context vào training chuẩn. Transformer decoder-only biến quyền truy cập đó thành biểu diễn sâu hơn qua nhiều block, nhưng yêu cầu nhiều compute, activation memory và KV cache hơn.

Kết luận quan trọng nhất về phương pháp là không được dùng một bảng số giả định để thay cho thí nghiệm. Cùng corpus, tokenizer, split, seed và token budget là điều kiện cần nhưng chưa đủ cho causal attribution. Báo cáo nên bổ sung parameter-matched và compute-matched tracks, tách quality khỏi systems metrics, đo effective context thay vì chỉ receptive field, và báo cáo độ bất định. Với protocol này, dự án MiniGPT có thể tạo ra một nghiên cứu nhỏ nhưng có giá trị: không chỉ tái hiện Transformer, mà giải thích có kiểm chứng từng cơ chế đã mua được bao nhiêu chất lượng và phải trả bao nhiêu chi phí.

A Công thức tham số tham khảo

Giả sử vocabulary V , hidden size d , max position L_{max} , MLP ratio 4, untied embeddings:

$$P_{\text{embed+head}} \approx 2Vd + V + L_{max}d, \quad (18)$$

$$P_{\text{attention}} \approx 4d^2, \quad (19)$$

$$P_{\text{MLP}} \approx 8d^2, \quad (20)$$

$$P_{\text{block}} \approx 12d^2 + O(d), \quad (21)$$

$$P_{\text{decoder}} \approx 2Vd + L_{max}d + n_{\ell}(12d^2 + O(d)). \quad (22)$$

Các con số chính thức phải lấy từ ‘sum(p.numel() for p in model.parameters())’, đồng thời tách embedding/non-embedding.

B Checklist tái lập

Mục	Bảng chứng cần lưu
Dữ liệu	URL/nguồn, license, checksum, preprocessing script, split manifest
Tokenizer	Model file, vocab file, config, checksum, training corpus hash
Code	Git commit, dirty status, submodule revisions
Môi trường	OS, Python, package lock, CUDA/cuDNN, driver, container digest
Hardware	GPU/CPU/RAM, số thiết bị, interconnect
Randomness	Tập seed, deterministic flags, DataLoader worker seeding
Training	Config YAML/JSON, logs theo token, checkpoint selection rule
Evaluation	Script version, batch, context, decoding config, metric definitions
Profiling	Warm-up, repetitions, synchronization, p50/p95, memory counters
Statistics	Seed-level values, document bootstrap, confidence intervals
Reporting	Raw CSV/JSON, script tạo bảng và hình, limitations

C Cấu trúc thư mục thí nghiệm đề xuất

```
minigpt-research/
  configs/
```

```
data.yaml
tokenizer.yaml
model_bigram.yaml
model_attention.yaml
model_transformer.yaml
data/
  manifests/
  checksums.txt
src/
  data/
  models/
  train/
  eval/
  profiling/
experiments/
  <run_id>/config.lock.yaml
  <run_id>/metrics.jsonl
  <run_id>/environment.txt
reports/
  tables/
  figures/
reproduce.sh
```

References

- [1] S. F. Chen and J. Goodman, “An Empirical Study of Smoothing Techniques for Language Modeling,” *Computer Speech & Language*, vol. 13, no. 4, pp. 359–394, 1999. DOI: [10.1006/csla.1999.0128](https://doi.org/10.1006/csla.1999.0128).
- [2] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, “A Neural Probabilistic Language Model,” *Journal of Machine Learning Research*, vol. 3, pp. 1137–1155, 2003. [JMLR paper page](#).
- [3] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems 30*, 2017. [NeurIPS paper page](#).
- [4] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training,” OpenAI, 2018. [Technical report](#).
- [5] A. Radford et al., “Language Models are Unsupervised Multitask Learners,” OpenAI, 2019. [Technical report](#).
- [6] J. Kaplan et al., “Scaling Laws for Neural Language Models,” arXiv:2001.08361, 2020. [arXiv](#).
- [7] J. Hoffmann et al., “Training Compute-Optimal Large Language Models,” arXiv:2203.15556, 2022. [arXiv](#).
- [8] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in *NeurIPS*, 2022. [arXiv](#).
- [9] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 2023. DOI: [10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165).
- [10] N. Shazeer, “Fast Transformer Decoding: One Write-Head is All You Need,” arXiv:1911.02150, 2019. [arXiv](#).
- [11] T. Kudo and J. Richardson, “SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing,” in *EMNLP System Demonstrations*, pp. 66–71, 2018. DOI: [10.18653/v1/D18-2012](https://doi.org/10.18653/v1/D18-2012).
- [12] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer Sentinel Mixture Models,” arXiv:1609.07843, 2016. [arXiv](#).
- [13] D. Paperno et al., “The LAMBADA Dataset: Word Prediction Requiring a Broad Discourse Context,” in *Proceedings of ACL*, pp. 1525–1534, 2016. DOI: [10.18653/v1/P16-1144](https://doi.org/10.18653/v1/P16-1144).
- [14] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *ICLR*, 2019. [arXiv](#).
- [15] K. Heafield, “KenLM: Faster and Smaller Language Model Queries,” in *Proceedings of the Sixth Workshop on Statistical Machine Translation*, pp. 187–197, 2011. [ACL Anthology](#).

- [16] PyTorch Contributors, “Reproducibility,” *PyTorch Documentation*, accessed June 19, 2026. [Documentation page](#).